

BISINDO Bridge: Lightweight Dual-Hand Landmark Recognition for In-Browser Sign Language Communication in Video Meetings

Felicia Gunardi¹, Jouxing Ngo¹, Kevin Pierre Rafael Sabran¹, Muhammad Naufal Dwitama¹, Wiby Aldiputra¹, and Gabriel Asael Tarigan¹

Computer Science Department, BINUS University, Jakarta, Indonesia

Abstract. Online meetings remain largely inaccessible for millions of deaf individuals in Indonesia — mainstream platforms provide little native support for Bahasa Isyarat Indonesia (BISINDO), and prior BISINDO recognition research is dominated by heavy sequence models over video that are rarely deployed as communication tools. We present **BISINDO Bridge**, an end-to-end **dual-hand** recognition system that runs **entirely in the browser** and integrates into a multi-party video meeting. Each gesture is encoded as an 84-dimensional vector (21 landmarks $\times$ 2 coordinates $\times$ 2 hands) from MediaPipe Hands, normalized in three stages for position and scale invariance, and classified by a compact **one-dimensional CNN** (61,274 parameters). On a dual-hand dataset of 26,192 samples (about 1,000 per letter; 1,192 for A), the deployed model achieves **98.45% accuracy** and 98.45% weighted F1 on a stratified 15% hold-out (3,929 samples). Converted to ONNX (241 KB), inference runs on-device via ONNX Runtime Web, so no video leaves the user’s browser. Beyond classification, we implement sentence composition from gesture delimiters and a **full-mesh WebRTC** meeting with live letter broadcasting over Google STUN and a best-effort public TURN fallback; the mesh imposes an $O(n^2)$ connection cost that we evaluate as a scalability limit. A single-hand ablation reaches 99.12% but cannot support the two-handed nature of most BISINDO letters and is not deployed. We frame the system as a strong, honestly-evaluated **proof-of-concept** rather than a production-ready deployment.

Keywords: BISINDO (Indonesian Sign Language) · Dual-Hand Gesture Recognition · MediaPipe · 1D-CNN · ONNX Browser Inference · WebRTC

1 Introduction

Communication underpins participation in education, work, and social life. For people with hearing and speech impairments in Indonesia, Bahasa Isyarat Indonesia (BISINDO) is a primary language, yet unfamiliar to most hearing people. Virtual meeting tools such as Google Meet and Zoom amplify this gap: they center spoken language and text chat, leaving deaf participants dependent on scarce human interpreters.

Two sign systems coexist in Indonesia. SIBI is the formally standardized system designed for consistency in instruction; BISINDO is the natural sign language that Deaf communities actually use in daily conversation, regional and continuously evolving. Crucially for recognition, the BISINDO manual alphabet is signed predominantly with *both* hands (only a few letters reduce to one), so any practical recognizer for the language people actually sign must accept two-hand input — not as an option, but as a requirement. Meeting platforms offer neither interpreters at scale nor any keypoint-based signing support, making the accessibility gap structural rather than cosmetic.

Computer vision can reduce that barrier. MediaPipe Hands [1] estimates 21 anatomical landmarks per hand on-device, enabling recognition from sparse keypoints rather than raw pixels. For BISINDO, prior work mostly uses sequence models (LSTM, CNN-LSTM) over video [2–4], which reach competitive offline accuracy but are heavy for browser deployment and seldom close the loop into a usable meeting product. Lightweight *single-frame* dual-hand classification, deployed as an accessible communication tool, remains underexplored.

This paper presents **BISINDO Bridge**, combining dual-hand landmark extraction, a compact 1D convolutional classifier, browser-based ONNX inference, and a WebRTC meeting with sentence building and live broadcasting (Fig. 2). Our contributions:

- a **dual-hand landmark pipeline**: $21 \times 2 \times 2 = 84$ features per frame with three-stage normalization;
- a **lightweight 1D-CNN** (61,274 parameters) trained on 26,192 samples, reaching **98.45%** accuracy and weighted F1 on 3,929 held-out samples, independently re-evaluated;
- **privacy-preserving browser deployment**: PyTorch→ONNX (241 KB) with numerical parity better than 4×10^{-6} and no server-side vision;
- an **end-to-end meeting product** (full-mesh WebRTC, sentence builder with hold gestures, TTS) with ICE-restart recovery, plus an honest evaluation of full-mesh scalability.

The remainder of this paper is organized as follows. Section 2 positions the work against prior BISINDO recognizers. Section 3 outlines the system and its design goals, and Section 4 details the dataset, normalization, and classifier. Section 5 covers the browser-side implementation, Section 6 reports results, Section 7 discusses limitations, and Section 8 concludes.

2 Related Work

2.1 Sign Language Recognition

Sign language recognition has moved through three input regimes. Early systems classify RGB frames directly with 2D CNNs; they reach high accuracy but inherit the full image as input, which makes them sensitive to background and lighting and expensive at the edge [5–7]. Skeleton-based methods replace pixels with tracked joint coordinates, cutting input dimensionality by orders of magnitude while keeping the pose structure that defines a sign; reviews of the field treat this representation as a standard route to real-time recognition [5, 7]. A third regime applies sequence models (LSTM, CNN–LSTM) over landmark or frame streams to capture temporal dynamics, at the cost of larger models and higher latency.

For BISINDO specifically, published systems cluster in the second and third regimes. Auziqni [2] combines MediaPipe Holistic with an LSTM and reports about 97% on 780 recorded sequences; evaluation is offline, on video clips rather than live input. Afwan et al. [3] classify skeletal features with an LSTM and report roughly 93% validation accuracy on alphabetic gestures. Aljabar and Suhajito [4] stack a CNN–LSTM hybrid over video. Fadzli and Rahardi [8] compare MobileNetV3 and EfficientNetV2B0 on MediaPipe landmark input with a mobile target. Three gaps separate them from a usable communication tool: accuracy is measured offline on pre-recorded input, the models target phones or standalone scripts rather than a browser, and none places recognition inside a live meeting where several people read the letters. This work differs on all three counts.

2.2 Hand Landmark Extraction

MediaPipe Hands [1, 9] detects up to two hands and returns 21 landmarks per hand (the wrist plus four joints on each finger), estimated on-device in real time on CPU. A raw frame at 640×480 carries roughly 9.2×10^5 pixel values; the landmark representation reduces the information relevant to hand pose to 42 or 84 numbers — about four orders of magnitude. The layout also mirrors how fingerspelling is articulated: each finger contributes one joint chain rooted at the shared wrist, so letter contrasts — a curled index versus an extended one, fingertips touching versus apart — appear directly as differences between adjacent landmark positions rather than as global image shifts. The sparse form is also robust to cluttered backgrounds and lighting shifts, and it lets the whole pipeline run inside a browser tab without pixels leaving the device. The trade-off is dependence on tracker quality: when the tracker misses a hand, the classifier sees an all-zero block, which our normalization leaves unchanged (Section 4.3).

2.3 One-Dimensional CNNs for Gestures

A 1D convolution slides over an ordered numerical sequence — sensor streams, audio, or a flattened landmark vector — and learns local patterns with far fewer parameters than an image CNN needs. Al Mudawi et al. [10] apply the idea to daily-life hand gestures from low-dimensional signals; Kumar et al. [11] combine MediaPipe output with CNN classifiers for real-time ASL. For landmark input the fit is structural: neighbouring positions in the vector are neighbouring joints, so a width-3 kernel covers each joint together with its immediate neighbours, without inventing the 2D spatial grid that flattened coordinates do not have. Stacking such kernels deepens the effective receptive field: a third layer sees each joint in the context of its neighbours’ neighbours, enough to capture the overall hand shape without ever forming an image. Because the alphabet task here is single-frame — every letter is a static posture — no recurrent memory is needed, so the classifier stays minimal and per-frame latency stays low. The resulting models are small enough to run where this paper needs them, inside the browser.

2.4 Research Gap

Two gaps motivate this work. First, BISINDO classification that runs in-browser on static frames is less studied than video-sequence models: the systems above assume recorded clips and none reports a browser-only deployment. Second, published work generally stops at accuracy tables. A fingerspelling aid is only useful inside the place where deaf participants actually communicate — the meeting itself — which requires combining recognition with sentence building, edit gestures, and multi-party video. We address both gaps with a deployed dual-hand 1D-CNN and a WebRTC collaboration stack, and we report the failure modes of the full system rather than classifier accuracy alone (Section 6.6). Figure 1 places this work against the systems above on the three axes where they diverge.

In short, the missing piece is not another recognizer but the bridge from one: a classifier small enough to run in every participant’s browser, paired with the meeting plumbing that turns recognized letters into participation — speech output, shared text, and sessions that survive network hiccups.

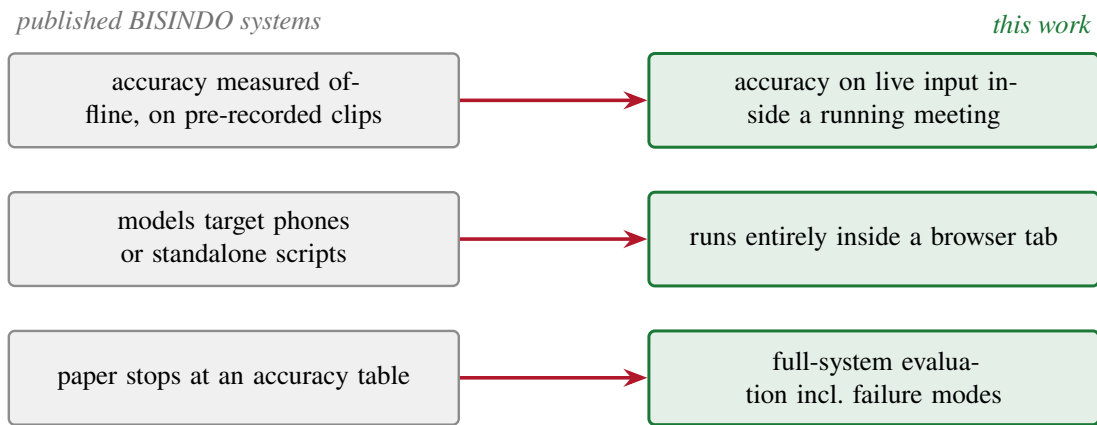


Fig. 1. Three gaps between published BISINDO recognition systems and a usable communication tool, and where **BISINDO Bridge** sits on each.

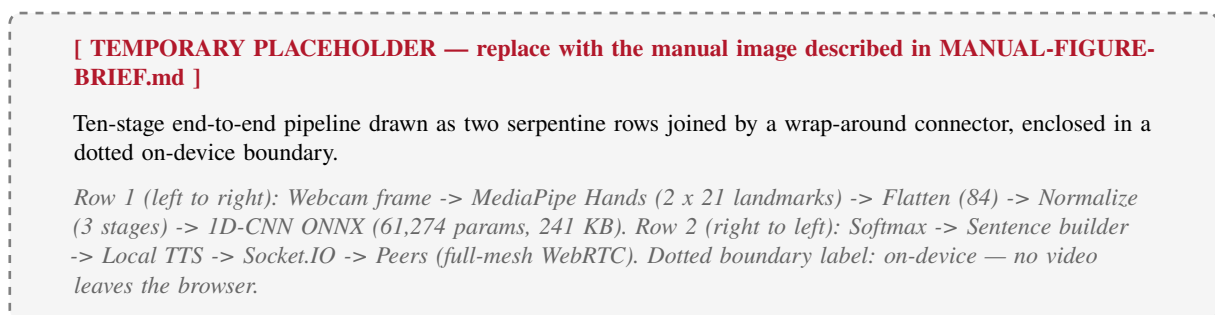


Fig. 2. End-to-end **BISINDO Bridge** pipeline: webcam capture, dual-hand landmark normalization, on-device ONNX 1D-CNN classification, sentence building, and full-mesh broadcast. All vision and classification stay in the browser; the server only signals and relays non-media events.

3 System Overview

Figure 2 summarizes the end-to-end path. A webcam frame is processed by MediaPipe Hands (up to two hands). Landmarks are flattened to 84 features, normalized, and classified by the ONNX 1D-CNN into a letter A–Z. A sentence builder commits stable letters, applies fist/palm edit gestures, and uses no-hand timers for word/sentence boundaries. Letters and sentences are spoken locally (TTS) and/or broadcast over Socket.IO to peers whose media flows over WebRTC. All vision and classification run on the client; the server only signals and relays non-media events. Figure 3 shows the resulting experience in a live call.

Three design goals follow from the meeting setting. First, *privacy by construction*: video frames never leave the browser tab, so the only traffic besides WebRTC media is short text events. Second, *latency as a feature*: a letter must appear while the conversation is still alive, which pushes the design toward single-frame classification (no temporal buffering) and a model small enough for ONNX Runtime Web. Third, *zero-install deployment*: participants join from a URL with no app install, which rules out native pipelines and motivates the browser-only stack.

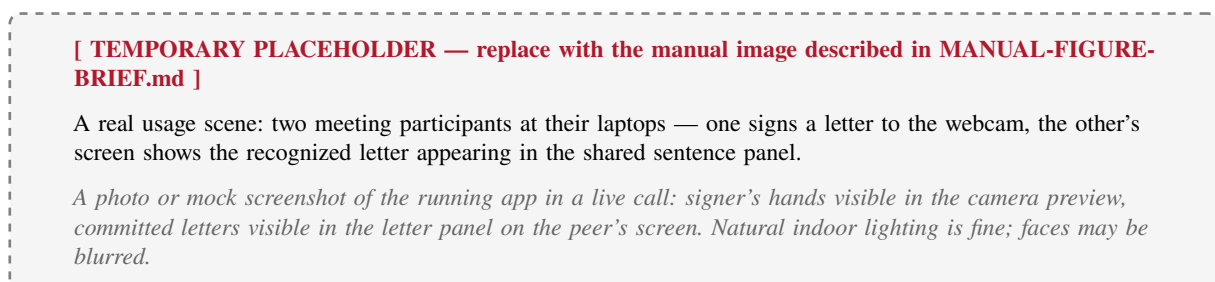


Fig. 3. **BISINDO Bridge** in context: one participant signs; the letter is recognized on-device and appears as text on every peer's screen, while full-mesh audio/video keeps the conversation natural.

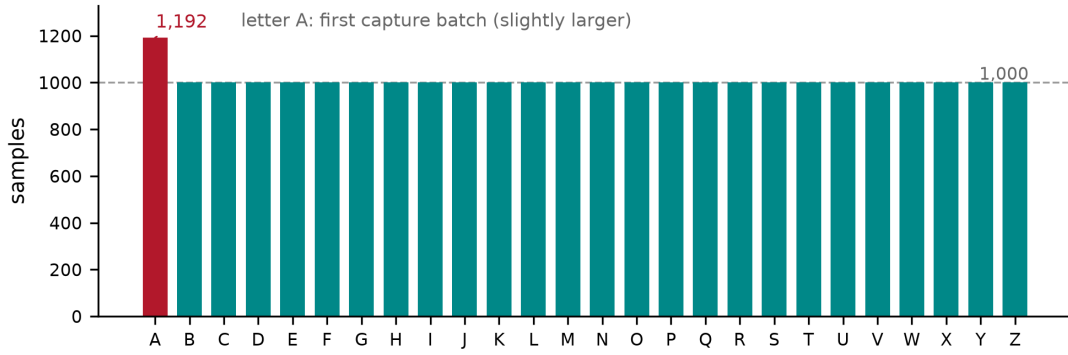


Fig. 4. Dataset composition: 26,192 dual-hand samples across 26 letters (letter A = 1,192 from the first capture batch; B–Z = 1,000 each). The stratified 85/15 split preserves these proportions in the 3,929-sample test set.

4 Methodology

4.1 Dual-Hand Dataset

The primary dataset is a dual-hand collection of 26,192 samples across 26 letters ($\approx 1,000$ per class), stored as 84-dimensional (x, y) landmark vectors captured via webcam with MediaPipe extraction and manual labels from nine project contributors. We use a stratified 85/15 train–test split (seed 42), yielding 3,929 test samples with class proportions preserved; only landmark coordinates (no raw images) are stored.

[TEMPORARY PLACEHOLDER — replace with the manual image described in MANUAL-FIGURE-BRIEF.md]

A 3x2 grid of six real two-hand frames from the corpus, one per letter (A–F), each annotated with its letter label.

Take six sample frames from the dataset (letters A to F), crop them to a consistent framing, arrange in 3 columns $\times$ 2 rows, and put a small letter badge in the corner of each tile. Same lighting/size across tiles.

Fig. 5. Sample frames from the dual-hand corpus (one per letter, A–F): two-hand alphabet signs captured frontally with a laptop webcam.

Figure 5 shows six representative corpus frames; Figure 4 shows the per-letter counts. Letter A carries 1,192 samples — the first capture batch ran slightly longer before the 1,000-per-letter quota was fixed — and every other letter carries exactly 1,000. A stored sample is one frame: 84 landmark numbers describing a single hand-pose snapshot, not a video clip. That granularity keeps the dataset compact and matches the single-frame recognizer the deployment needs; the stratified split then preserves these proportions in the 3,929-sample test set, so no letter is judged on a token sample.

The set is modest for production (captured by a small, homogeneous group under similar conditions), so we regard the model as a strong proof-of-concept, *not yet production-ready* (Section 7). For a short ablation only, we also reference single-hand variants (63 features (x, y, z) from one hand) derived from a larger canonical capture stream; they are *not* deployed and only contextualize the accuracy–usability trade-off in Section 6.4.

4.2 Landmark Extraction

MediaPipe Hands [1] returns 21 landmarks per hand. Analysis of the BISINDO alphabet shows 15 of 26 letters (A, B, D, F, G, H, K, P, Q, R, S, T, W, X, Y) are predominantly two-handed, and the remaining 11 (C, E, I, J, L, M, N, O, U, V, Z) single-handed. This grouping is practical rather than purely linguistic: a few letters (e.g., M, N, Q) are treated as single-handed because MediaPipe’s front-camera 2D view cannot reliably disambiguate their two-handed articulations, so we adopt the representation the tracker captures consistently. The system requests up to two hands. For dual-hand features we keep (x, y) and drop depth z , which is noisier across hands and cameras; a missing second hand is zero-padded to a fixed length of 84. Figure 6 plots two tracked hands from the corpus.

The landmark set is anatomically ordered: landmark 0 is the wrist, 1–4 run along the thumb, 5–8 the index finger, 9–12 the middle, 13–16 the ring, and 17–20 the little finger, so consecutive vector positions are physically adjacent joints — exactly the locality the width-3 convolution exploits later (Section 4.4). Keeping (x, y) per joint and dropping the noisy depth axis gives 42 numbers per hand; a two-handed letter fills both 42-slot blocks and a single-handed sign leaves the second block zero-padded.

[TEMPORARY PLACEHOLDER — replace with the manual image described in MANUAL-FIGURE-BRIEF.md]

Two real tracked hands with the MediaPipe skeleton overlay: 21 numbered landmarks per hand, wrist in red, fingertips in orange, connected in skeleton order.

Best source: screenshot of the running app with the hand-overlay preview visible, signing a two-handed letter; tight crop, light background, at least 1600 px wide. Alternative: any MediaPipe Hands debug screenshot with two hands visible.

Fig. 6. Landmark extraction on two real tracked hands from the corpus: 21 anatomical landmarks per hand (wrist in red, fingertips in orange), connected in skeleton order. Each hand yields 42 (x, y) numbers; two hands yield the 84 features used by the recognizer.

[TEMPORARY PLACEHOLDER — replace with the manual image described in MANUAL-FIGURE-BRIEF.md]

The same two-hand frame shown three times side by side: raw, after wrist translation, after hand-size scaling.

Left panel: raw hands at different image positions and apparent sizes. Middle panel: both wrists pinned to the origin. Right panel: both hands at identical extent. Keep one hand teal and the other blue throughout; a small note mentions stage 3 (per-axis z-score).

Fig. 7. The first two normalization stages applied to one real two-hand frame (teal = hand 1, blue = hand 2): raw hands differ in position and apparent size; wrist translation pins both wrists to the origin; hand-size scaling gives both hands identical extent. Stage 3 (per-axis z-score) then standardizes each axis using training-split statistics only.

4.3 Three-Stage Normalization

Raw coordinates depend on image position and camera distance. We apply (Fig. 8):

1) Wrist translation. For each hand, subtract the wrist (landmark 0):

$$p'_i = p_i - p_0. \quad (1)$$

2) Hand-size scaling. Divide by the maximum Euclidean distance from the wrist to any landmark of the same hand:

$$\hat{p}_i = \frac{p'_i}{d_{\max}}, \quad d_{\max} = \max_j \|p_j - p_0\|. \quad (2)$$

3) Standardization. A StandardScaler fitted on the training split maps features to zero mean and unit variance; mean/scale vectors are saved with the model for identical inference transforms. Zero-padded absent-hand slots are left unchanged through normalization. Figure 7 shows the first two stages on one real two-hand frame.

The raw hands sit at different image positions and different apparent sizes — depending on where, and how far from the camera, each signer holds their hands. Translation pins both wrists to the origin and size scaling gives both hands the same extent, after which the geometric details that carry no sign information are gone; stage 3 then folds each axis into zero-mean, unit-variance space using statistics fitted on the training split only.

4.4 Feature Representation

After normalization, landmarks flatten to

$$\mathbf{x} \in \mathbb{R}^{84} = [\text{hand}_1(x, y)_{0..20}; \text{hand}_2(x, y)_{0..20}]. \quad (3)$$

This 84-vector is the sole input of the deployed model. The slot order is fixed: landmarks 0–20 of the first detected hand occupy positions 0–41 and those of the second hand positions 42–83, in the same joint order every frame. Fixed slots make zero-padding meaningful — a missing hand contributes an all-zero block that normalization leaves untouched — and give the convolution a predictable geometry: a width-3 kernel reads each joint with its immediate neighbours inside one hand, except at the two block-boundary positions where kernels see both hands. Single-hand ablations use $\mathbb{R}^{63}$ with (x, y, z) on one hand (Section 6.4). Figure 9 unpacks the vector.

The 84 slots form two fixed 42-slot blocks, one per hand, in a joint order that never changes between frames; the fixed semantics are what make zero-padding meaningful and give the convolution a predictable neighbourhood. A kernel centred on an interior joint reads that joint and its two neighbours within the same hand; only at the single boundary between the blocks can a kernel reach across hands. When only one hand is present, the second block stays all zero through normalization, so the model sees an unambiguous “absent” pattern rather than noise.

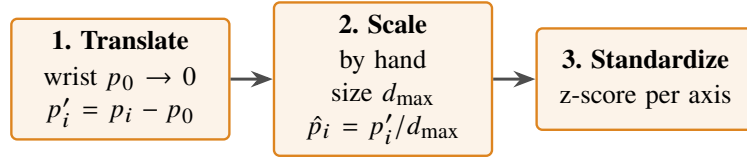


Fig. 8. Three-stage normalization: wrist translation, hand-size scaling, and per-axis z-score standardization.

[**TEMPORARY PLACEHOLDER** — replace with the manual image described in **MANUAL-FIGURE-BRIEF.md**]

A flat 84-slot strip in two colored blocks: hand 1 (slots 0–41, teal) and hand 2 (slots 42–83, blue), one cell per landmark.

Tick marks separate the cells; a dashed red line marks the block boundary between slots 41 and 42; one width-3 kernel window is highlighted inside hand 1 (joint + 2 neighbours, same hand), and the kernel crossing the boundary is marked as the only one that spans both hands.

Fig. 9. Anatomy of the 84-D input vector: two fixed 42-slot blocks, one per hand, in a joint order that never changes between frames. A width-3 kernel reads each joint with its two neighbours inside one hand; only at the block boundary can a kernel span both hands.

4.5 1D-CNN Architecture

We use a three-block 1D-CNN suited to ordered landmark vectors (Table 1). Adaptive average pooling to length 1 yields a fixed 64-D code regardless of input length, so the same head serves dual-hand inputs. Total trainable parameters: 61,274. Figure 10 traces the layer chain and the shrinking tensor length through each pooling stage.

[**TEMPORARY PLACEHOLDER** — replace with the manual image described in **MANUAL-FIGURE-BRIEF.md**]

Left-to-right layer chain: Input $1 \times 84 \rightarrow$ Conv1 (64, $k=3$, ReLU, Pool 2 $\rightarrow 64 \times 42$) $\rightarrow$ Conv2 (128, $k=3$, ReLU, Pool 2 $\rightarrow 128 \times 21$) $\rightarrow$ Conv3 (64, $k=3$, ReLU, AAP(1) $\rightarrow 64 \times 1$) $\rightarrow$ Flatten 64 $\rightarrow$ FC1 (64 $\rightarrow$ 128, ReLU, Drop 0.3) $\rightarrow$ FC2 (128 $\rightarrow$ 26) $\rightarrow$ Softmax A–Z.

Draw the boxes in ONE straight row, simple arrows between them, no wrap-around connectors. Above the conv boxes add a shrinking bar ruler labelled: spatial length $84 \rightarrow 42 \rightarrow 21 \rightarrow 1$.

Fig. 10. Deployed **1D-CNN** architecture (61,274 parameters): three conv blocks with max-pooling, then **adaptive average pooling** to a fixed 64-D code and two FC layers to 26 letter logits. Tensor length halves at each pooling stage (ruler above).

Single-hand ablation checkpoints that report higher numeric accuracy used a wider adaptive pool (flatten 512) and are not part of the deployed stack.

4.6 Training Protocol

Optimization uses Adam (learning rate 0.001), batch size 256, 50 epochs, and cross-entropy loss; dropout is 0.3 and the scaler is fit only on the training split. Artifacts saved per run: PyTorch weights, scaler JSON, label map, and metrics JSON. Unless noted, deployed-model metrics are confirmed by independent re-evaluation from the checkpoint with the same split seed. Figure 11 separates the two phases.

Everything expensive happens once, offline: fitting the scaler, optimizing the 61,274 parameters, and exporting the graph. The online lane repeats only cheap per-frame work — landmark extraction, the fixed normalization transform, and a 0.53 ms ONNX forward pass — inside the participant’s browser tab. This split is why a meeting stays live while recognition runs: no per-frame work depends on the training pipeline.

5 Implementation

5.1 ONNX Browser Inference

The dual-hand PyTorch checkpoint is exported with `torch.onnx.export` (opset 12, dynamic batch). The artifact is 241 KB on disk (247,229 bytes) with input shape [batch, 1, 84] and 26-way output. In the browser we load ONNX

Table 1. Deployed 1D-CNN (v1) for dual-hand 84-D input

Layer	Configuration
Input	(1, 84) after channel reshape
Conv block 1	64 filt., $k=3$, pad 1, ReLU, MaxPool 2
Conv block 2	128 filt., $k=3$, pad 1, ReLU, MaxPool 2
Conv block 3	64 filt., $k=3$, pad 1, ReLU, AdaptiveAvgPool(1)
Flatten	64
FC 1	$64 \rightarrow 128$, ReLU, Dropout 0.3
FC 2	$128 \rightarrow 26$ logits
Inference	Softmax over A–Z

[TEMPORARY PLACEHOLDER — replace with the manual image described in MANUAL-FIGURE-BRIEF.md]

Two horizontal lanes — OFFLINE (once, at training time) and ONLINE (every frame, in-browser) — connected by one dashed cross-lane arrow.

Offline lane: dual-hand dataset 26,192 -> 3-stage normalization (fit scaler) -> train 1D-CNN (61,274 params, 152.8 s CPU) -> export ONNX 241 KB. Online lane: MediaPipe Hands -> normalize (saved scaler) -> ONNX Runtime Web (0.53 ms mean) -> letter + sentence builder. Dashed arrow from ONNX export down into ONNX Runtime, labelled: same weights + scaler.

Fig. 11. Two phases of the deployed system: offline training (once) and online inference (every frame, in-browser). Only the green lane runs while a meeting is live.

Runtime Web [12] version 1.17.1, apply the same three-stage normalization via the saved scaler, and run softmax client-side. Numerical parity versus PyTorch was verified with maximum absolute difference below 4×10^{-6} . No frame or landmark leaves the device for classification, supporting privacy-sensitive use. Figure 12 traces what crosses the network and what does not.

[TEMPORARY PLACEHOLDER — replace with the manual image described in MANUAL-FIGURE-BRIEF.md]

On-device recognition box (dotted boundary: camera -> MediaPipe -> ONNX) versus the server and peer browsers, with exactly two outbound arrows.

Dashed arrow 1: letters/sentences (text events) from browser to Socket.IO server. Arrow 2: audio/video peer-to-peer from browser to peer browsers, bypassing the server. Small label under the browser box: 84 floats stay on-device.

Fig. 12. Network data flow. Recognition consumes 84 landmark numbers and runs entirely on-device; the signaling server relays text events only, and audio/video flows peer-to-peer over WebRTC without a media relay.

A 640×480 frame carries roughly 9.2×10^5 pixel values, yet recognition needs only 84 landmark numbers, and those never leave the tab. What does leave is the composed text — committed letters and sentences over Socket.IO — and the audio/video streams, which flow peer-to-peer over WebRTC. The signaling server therefore never touches a frame, a landmark, or a media packet; it only helps browsers find each other.

5.2 Sentence Builder

Letters are not emitted every frame. A client-side builder commits a letter only after the predicted class stays stable (default 500 ms). Edit gestures use a hold threshold (default 2 s): a closed fist deletes the last committed letter (backspace), an open palm clears the current word. Word and sentence boundaries follow no-hand timers (default 2 s for space, 4 s for sentence completion). Completed text can be spoken with browser TTS (id-ID) and emitted as Socket.IO events (letter_committed, space_inserted, sentence_completed) for remote display. Figure 13 summarizes the rule set.

A short walkthrough shows the defaults working together. The signer holds B long enough to commit it, then I and N; a brief misread of U is removed with a two-second fist; an open palm clears the current word if needed; two seconds of empty hands insert a space, and four seconds end the sentence, which TTS then speaks aloud. All thresholds are configuration constants rather than learned parameters, so tuning latency against misfire rate is a one-line change, not a retraining cycle.

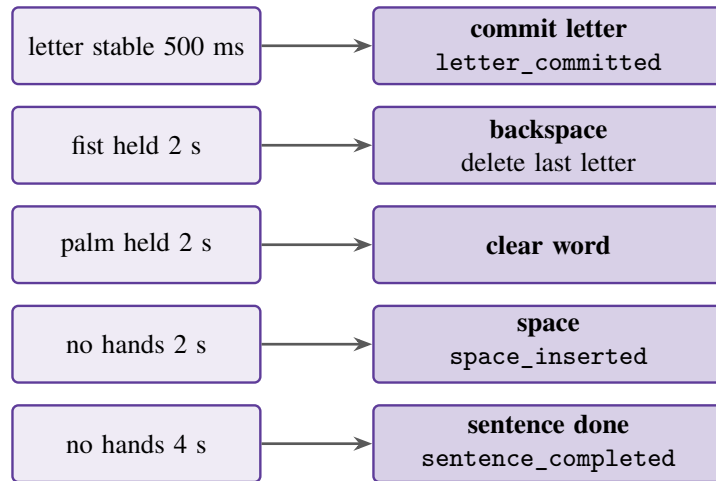


Fig. 13. Sentence-builder rules: stability commit, fist/palm edit gestures, and no-hand timers for word and sentence boundaries.

5.3 Meeting Architecture

Figure 14 shows the collaboration design. Media flows full-mesh over WebRTC among peers, simple to deploy but poorly scaling: a room of n participants requires each peer to maintain $n - 1$ outbound media streams, an $O(n^2)$ total connection cost, acceptable for small rooms but a bottleneck as n grows (Section 6.5). The Flask-SocketIO server performs room membership and SDP/ICE signaling only; it does not relay video. Google STUN is always available for NAT traversal; a public best-effort TURN (Metered openrelay) serves as a client-side fallback, and an optional self-hosted TURN can be supplied at runtime via `/api/turn` so credentials are never hard-coded in static assets. Our deployment provisions no dedicated TURN (the host sits behind consumer NAT), so we rely on the public STUN/TURN path; a media-relaying SFU is future work.

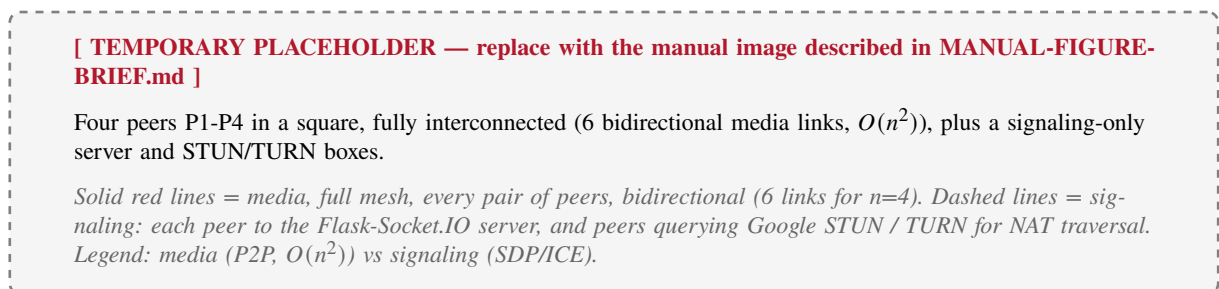


Fig. 14. Meeting architecture: bidirectional full-mesh WebRTC media ($O(n^2)$; six links for $n=4$) with a **signaling-only** Flask-Socket.IO server; peers query STUN/TURN for NAT traversal.

5.4 Reliability Fixes

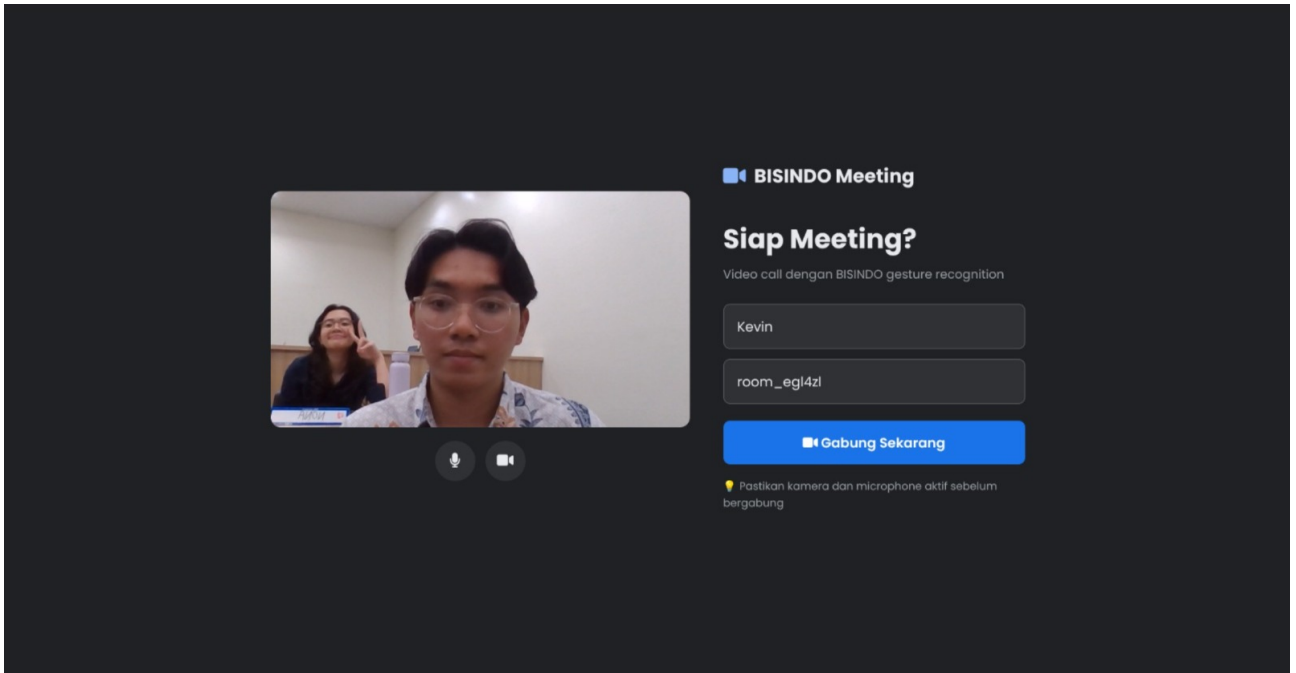
Early multi-peer tests exposed three failure modes, each addressed:

- **Black/partial video under restrictive NAT** (CGNAT, campus WiFi; STUN-only sessions) → public best-effort TURN fallback.
- **Video dead after transient ICE drops** → automatic ICE restart with a guarded full reconnect on failure.
- **Join races leaving existing peers uncontacted** → proactive connections from each joiner to every existing member, with deterministic offer polarity by session id.

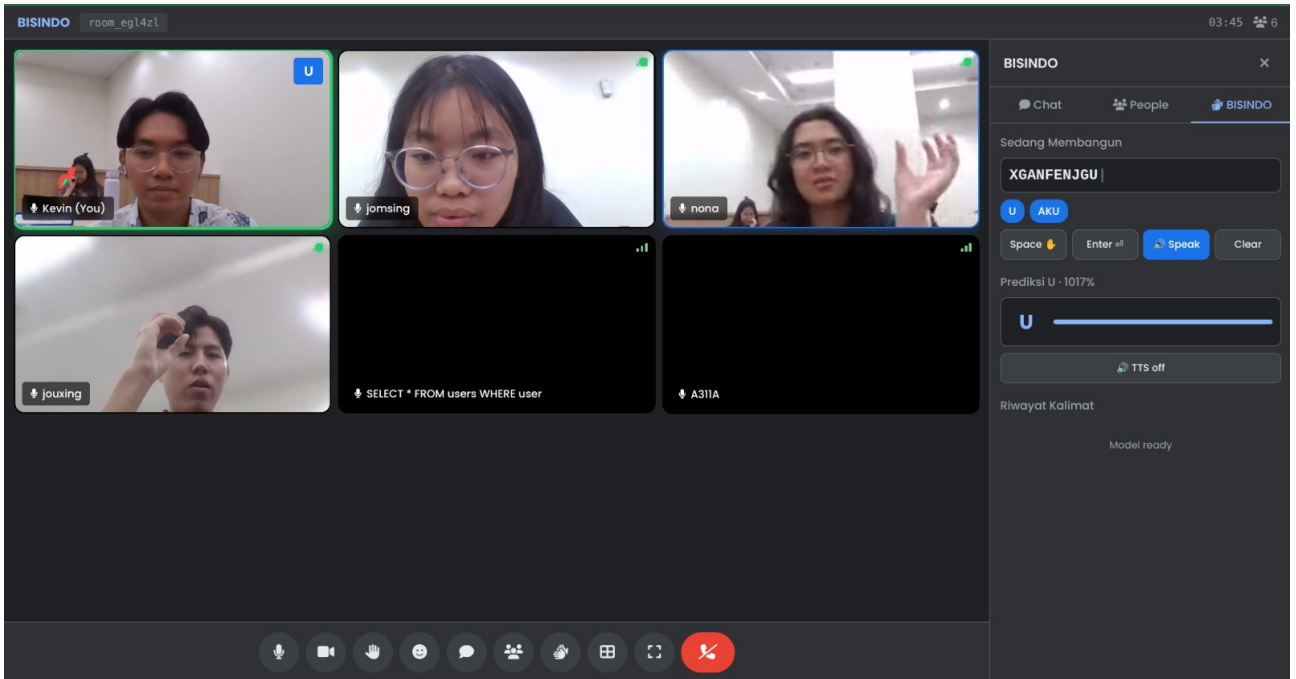
These fixes improve connection establishment and recovery in small internal sessions but do *not* remove the fundamental mesh scaling limit: each peer uploads a separate stream to every other peer, so sessions degrade asymmetrically as the room grows (Section 6.5). Automated HTTPS smoke checks against the deployed tunnel return HTTP 200 for the meeting shell, health/stats APIs, ONNX assets (model.onnx 247,229 B, scaler, labels), and client scripts.

5.5 Deployment Footprint

A static web demo (capture and offline test pages) suits static hosting (e.g., Vercel). The meeting service runs as a lightweight local Flask-SocketIO process on a commodity laptop (Intel Core Ultra 5 125U, 14 logical cores, no GPU);



(a). Pre-join lobby with camera preview.



(b). Live letter composition panel.

Fig. 15. Client UI: (a) pre-join lobby; (b) live letter composition with confidence readout.

public HTTPS ingress uses a Cloudflare quick tunnel, so the host need not expose inbound media ports or a public IP. Quick-tunnel URLs are ephemeral and change on restart; a named tunnel is planned for stable public demos. The small footprint follows from the architecture rather than from a post-hoc optimization: the model ships inside the web bundle, so the server process holds no weights and spends its budget on signaling alone, and the $O(n^2)$ media fan-out of the mesh lands on the participants' uplinks rather than on the host (Section 6.6). The same commodity laptop serves training, inference testing, and the meeting itself, which is what makes single-machine demonstrations and repeated redeployment during development practical.

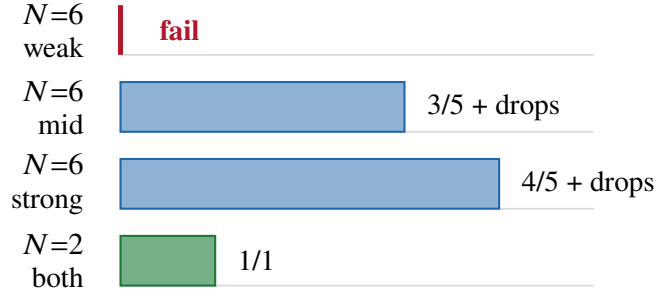


Fig. 16. Full-mesh reception from Table 4: remote streams rendered (of $N-1$) per observer in the live session. Reception is asymmetric and collapses entirely on the weak (cellular) uplink.

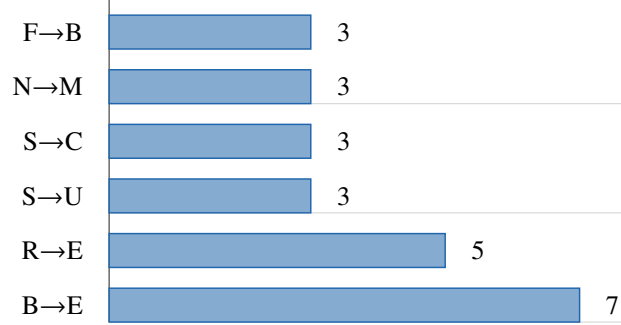


Fig. 17. Most frequent misclassifications of the dual-hand model (true → predicted, test set $n=3,929$).

6 Experiments

6.1 Setup

Training used PyTorch on CPU (Intel Core Ultra 5 125U, 14 logical cores, 30 GB RAM; no GPU) with the protocol in Section 4.6; dual-hand training completed in 152.8 s wall time. Metrics are accuracy, weighted F1, and per-class precision/recall/F1 (scikit-learn). The dual-hand test set has 3,929 samples (stratified 15%, seed 42). Deployment checks used Chrome with MediaPipe Hand Landmarker (float16) and ONNX Runtime Web 1.17.1. The meeting service runs as a local Flask-SocketIO process (signaling only) over a Cloudflare quick tunnel; recognition stays on-device in each browser.

6.2 Evaluation Metrics

We report accuracy, weighted F1, and per-class precision, recall, and F1, computed with scikit-learn on the held-out test set ($n=3,929$). Accuracy alone can hide class-level failures: a model can be 98% correct and still miss one letter consistently, which matters here because every wrong or missed letter enters the sentence the deaf participant is composing. Per-class precision (how often a predicted letter is right) and recall (how often a true letter is caught) expose exactly that. Weighted F1 averages per-class F1 with support as the weights; because the stratified split keeps classes nearly equal at about 150 test samples per letter, weighted and macro F1 nearly coincide here, and the weighted F1 (98.45%) matching the accuracy confirms the model does not lean on frequent classes. The classifier head is a softmax over the 26 letters, trained with categorical cross-entropy.

6.3 Main Result: Dual-Hand Model

The deployed dual-hand model reaches **98.45% accuracy** and **98.45% weighted F1**, independently re-evaluated from the saved checkpoint: 3,868 of 3,929 held-out samples are correct, and the 61 residual errors cluster on visually similar letters (Fig. 17; full matrix in Fig. 19). Full per-class metrics are in Table 2 and Fig. 18; the hardest letters are B (0.94 F1), E (0.96), and F (0.97), consistent with overlapping finger configurations. The floor matters as much as the average: every one of the 26 letters scores at least 0.94 F1, so no letter fails consistently — the failure mode that would silently corrupt every word containing it. Expressed differently, about one test frame in 65 is misclassified, and the residual errors concentrate on letter pairs a human observer would also confuse rather than being spread over random classes.

6.4 Compact Ablation: Why Not Single-Hand?

Table 3 contrasts the deployed dual-hand model with two single-hand references trained on the larger canonical capture stream. The balanced single-hand model posts the highest number, 99.12%, but the comparison is not as simple as the

Table 2. Per-class performance – dual-hand model (re-evaluated, $n=3929$)

	L	P	R	F1	Sup	L	P	R	F1	Sup
A	.994	.994	.994	179	N	.980	.967	.973	150	
B	.952	.933	.943	150	O	.993	.993	.993	150	
C	.974	.993	.984	150	P	.980	1.00	.990	150	
D	1.00	.980	.990	150	Q	.993	1.00	.997	150	
E	.926	1.00	.962	150	R	.993	.967	.980	150	
F	.986	.953	.970	150	S	1.00	.960	.980	150	
G	.993	1.00	.997	150	T	.993	.993	.993	150	
H	1.00	1.00	1.00	150	U	.955	.993	.974	150	
I	.961	.980	.970	150	V	1.00	.987	.993	150	
J	.993	1.00	.997	150	W	.993	1.00	.997	150	
K	1.00	.987	.993	150	X	.987	.973	.980	150	
L	1.00	1.00	1.00	150	Y	.987	.987	.987	150	
M	.967	.973	.970	150	Z	1.00	.980	.990	150	

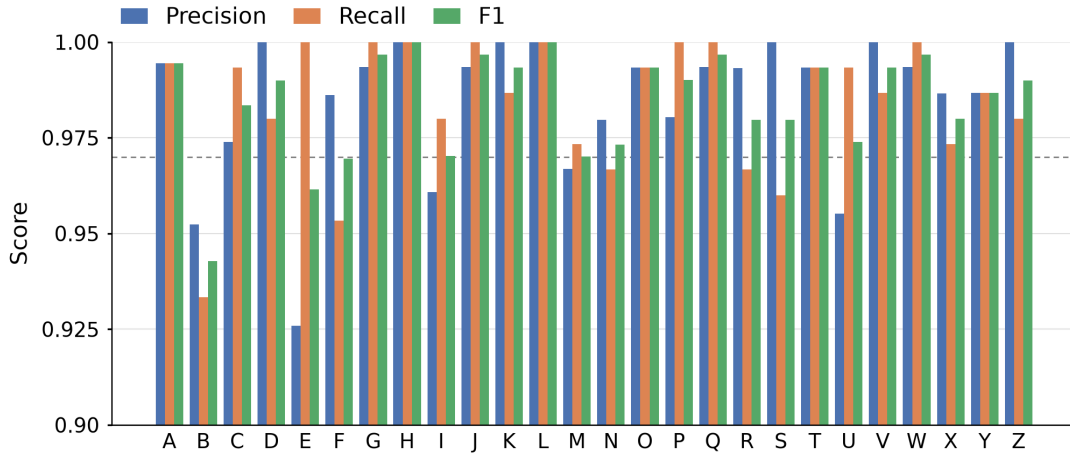


Fig. 18. Per-class precision, recall, and F1 (dual-hand model). Most letters exceed 0.97 F1; hardest are B, E, and F.

numbers suggest. A single-hand model sees one hand, so letters that require both — most of the BISINDO alphabet in our grouping (Section 4.2) — fall outside what it can ever recognize; its 99.12% applies to that restricted input world. The deployed dual-hand model accepts both hands and covers all 26 letters in one model, at a cost of 0.67 percentage points against the balanced reference. For a communication tool that must accept whatever a signer does, coverage is the point, so we deploy the dual-hand model and report its honest 98.45%.

6.5 Runtime and Product Checks

The deployed classifier is small (61,274 parameters, 241 KB ONNX), so classifier-only inference runs far above real-time; the full in-browser pipeline is dominated by MediaPipe landmark detection, not ONNX inference. Over 2,000 single-vector ($1 \times 1 \times 84$) inferences with ONNX Runtime on CPU (14 threads available), classifier-only latency is **0.53 ms mean / 0.31 ms median / 1.21 ms P95** ($\approx 1,870$ inferences/s), confirming the recognition head is not the bottleneck. Figure 20 breaks the per-frame cost down and summarizes the calibrated frame rate measured in Section 6.6.

The meeting stack is served locally behind a Cloudflare quick tunnel; public HTTPS checks return `status=ok`, serve the dual-hand ONNX bundle (241 KB), and load client scripts. Functional multi-user sessions confirmed the recognition overlay, live sentence broadcast, and recovery after transient network blips following the reliability fixes in Section 5. Figure 15 shows the pre-join lobby and the in-meeting recognition panel composing letters live. No formal deaf-community user study is claimed here.

6.6 System Evaluation (Mesh Scalability)

Because media uses a full mesh, per-peer upstream load grows with room size. To characterize the deployed system’s practical ceiling, we ran controlled multi-participant sessions, recording for each observer how many of the other $n - 1$ remote streams rendered and whether frame drops occurred (Table 4, Fig. 16). Reception is *asymmetric*: participants on stronger uplinks receive all peers, while those on weaker links receive only a subset, with degradation pronounced beyond a small room size. Concretely (Table 4): with two participants the mesh behaves like a direct call and everyone receives

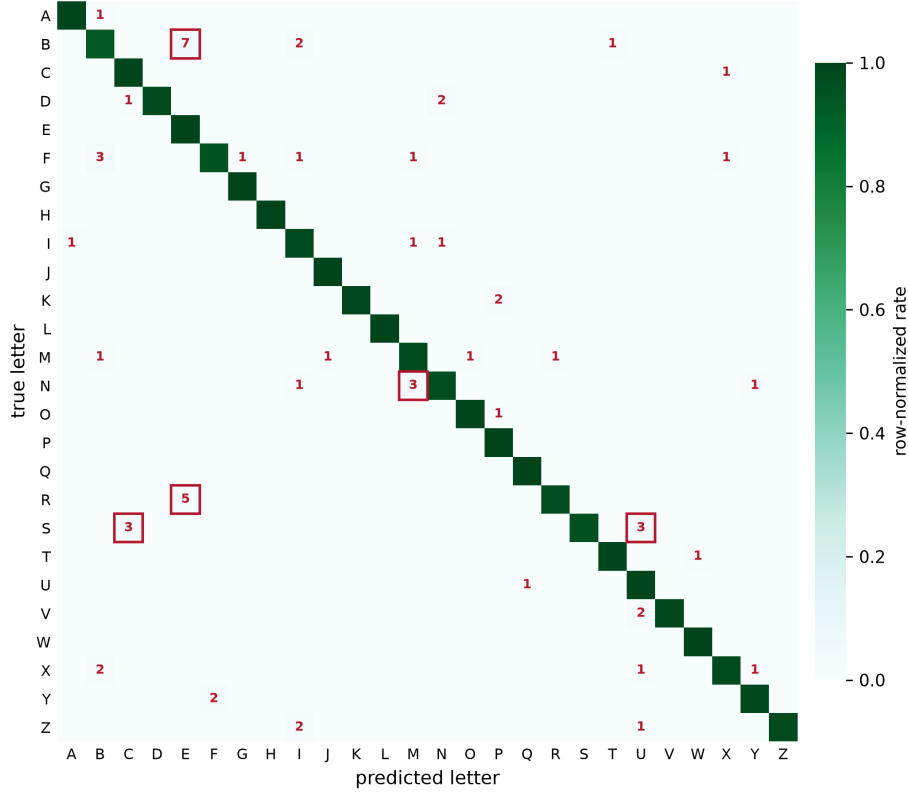


Fig. 19. Confusion matrix of the deployed dual-hand model ($n=3,929$, row-normalized shading; off-diagonal counts annotated). Only 61 of 3,929 frames are misclassified; the five most confusable pairs ($B \rightarrow E$, $R \rightarrow E$, $S \rightarrow U$, $S \rightarrow C$, $N \rightarrow M$, boxed) account for 21 of them and are all visually similar handshapes.

Table 3. Ablation: single-hand vs. deployed dual-hand

Model	Feat.	Hands	Acc.	Note
Baseline (1H)	63	1	96.04%	ref.
Balanced (1H)	63	1	99.12%	best acc.†
2Hand	84	2	98.45%	deployed‡

† training-time metric; not deployed (1-hand only).

‡ independently re-evaluated.

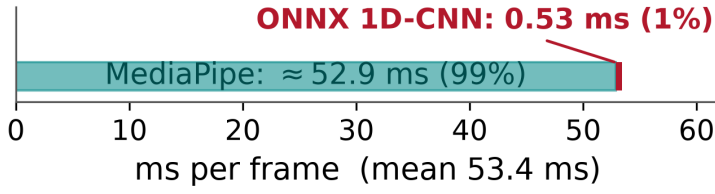
everything; at six, a strong-uplink observer still renders 4 of 5 streams, a mid-link observer 3 of 5, and a cellular-uplink observer fails to join usefully at all. The gap between strong and weak participants is the practical cost of $O(n^2)$ media without a relay, and it is the single clearest argument for the SFU listed as future work.

At $N = 2$ the mesh connects reliably, with both peers receiving audio and video and no drops. Larger rooms degraded clearly. In a live session peaking at six concurrent participants (of seven to eight who attempted to join), not all participants could see each other: reception was asymmetric; some observers rendered four of five remote streams, others three, at least one peer could not join, and some videos froze mid-session. These symptoms follow directly from the $O(n^2)$ full-mesh fan-out, where each peer uploads to every other. The limiting factor as rooms grow is upstream bandwidth and connection setup, not recognition, and weaker (mobile/cellular) uplinks fail first, which motivates the SFU/relay direction in future work.

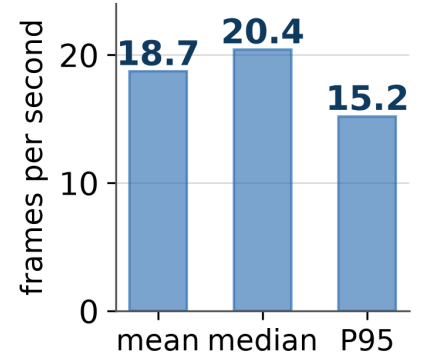
Qualitative evidence. Figure 21 shows the four failure modes live. The roster in Fig. 21b also reveals an unsanitized SQL-injection-style username stored and rendered verbatim (see limitations).

End-to-end latency and frame rate. We instrumented the in-browser pipeline (MediaPipe detection + normalization + ONNX inference + UI update) with `performance.now()` over 100 consecutive frames with a hand in view on a laptop with Intel integrated graphics (Figure 20): **18.7 FPS mean / 20.4 median** (15.2 at the P95-worst frame), i.e. **53.4 ms mean / 48.9 median** (65.7 ms P95). With classifier-only inference at 0.53 ms, essentially all per-frame cost is MediaPipe detection and rendering; recognition is not the bottleneck, and a steady ≈ 20 FPS suffices for the static-posture alphabet task.

Broadcast latency. We timestamped each letter-commit emit and measured the delay until it rendered on a remote peer over a live session ($n = 190$ committed letters). Because the clocks were not NTP-synchronized, absolute mean/min values are dominated by skew and meaningless (the raw mean was negative); we therefore report the *median*, robust to a constant offset: **59 ms median end-to-end**, with a P95 of ≈ 1.6 s from occasional tunnel/network stalls. The sub-100 ms



(a). Per-frame cost (53.4 ms mean).



(b). Frame rate (FPS).

Fig. 20. Browser pipeline latency and frame rate (100 frames, 640×480): ONNX inference is $\approx 1\%$ of a frame; MediaPipe landmark detection dominates.

Table 4. Full-mesh reception vs. room size (live observation). “Recv.” counts how many of the $N-1$ remote streams rendered; — = not recorded (the weak peer failed to join).

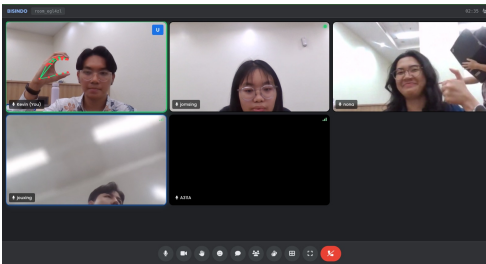
N	Observer	Recv.	Drops	Uplink
2	both	1/1	no	laptop / Wi-Fi
6	strong	4/5	yes	laptop / Wi-Fi
6	mid	3/5	yes	mixed
6	weak	fail	—	mobile / cellular



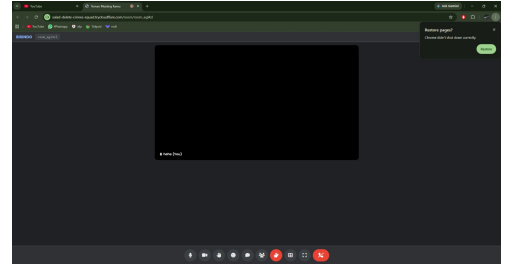
(a). Baseline: all tiles render.



(b). Roster reports 6, only 4 render.



(c). Black / frozen tiles.



(d). Joined room, blank stage.

Fig. 21. Live failure modes in a six-participant session: (a) baseline; (b) six in roster, four rendered; (c) black/frozen tiles; (d) blank stage after join.

median shows socket relay to peers is effectively interactive once a letter is committed; the heavy tail reflects the best-effort public tunnel, not the recognition or messaging path. A clock-synchronized re-measurement would tighten the absolute figures.

7 Discussion and Limitations

Accuracy vs. usability. The 99.12% single-hand score is a useful upper reference under idealized one-hand input, but a communication tool for BISINDO must accept two hands. The dual-hand model is therefore the system of record despite a small accuracy gap.

Table 5. Limitations of the current system.

Limitation	Detail
Data scale & diversity	$\approx 1,000$ samples/letter from a small, homogeneous group; strong held-out accuracy but too narrow for production—proof-of-concept, likely to scale with more data.
Signer concentration	Few contributors dominate capture; style bias.
Capture conditions	Front webcam, indoor, limited viewpoints; M, N, Q mapped to single-handed forms.
Evaluation rigor	Single 85/15 split, possible same-signer leakage; 98.45% likely <i>signer-dependent</i> ; leave-one-signer-out is the immediate next measurement.
Vocabulary scope	26 alphabet letters with rule-based delimiters, not full lexical BISINDO.
Transport & scale	Full mesh $O(n^2)$, best-effort public TURN (no SFU); quick-tunnel URLs suit demos, not production.
UI robustness	Occasional malformed confidence readout (“1017%”); display-only, classification unaffected.
Input hardening	Unsanitized display names (SQL-injection-style string stored verbatim); validation required before production.

Landmarks vs. pixels. Eighty-four floats per frame are enough for alphabet discrimination while remaining edge-friendly, aligning with the broader shift toward skeleton-based SLR [5].

What the failure modes teach. The live evaluation is as informative as the accuracy table. A room can be healthy for one participant and broken for another on the same network; tiles can go black or blank while the roster still counts them; and none of this is visible in an offline metric. The practical consequences are concrete: keep rooms small, monitor *rendered*-stream counts rather than roster counts, and treat a media-relaying SFU as the scaling path — all of which shape the future work below.

Threats to validity. Three caveats temper the numbers. First, the 85/15 split is signer-overlapping, so the reported accuracy is *signer-dependent*; a leave-one-signer-out protocol is the honest next measurement. Second, all captures share one webcam geometry, one indoor lighting profile, and a small contributor pool, so accuracy on unseen rooms, lighting, and skin tones is unmeasured. Third, latency and frame rate were profiled on a single mid-range laptop CPU; weaker devices will trade frame rate rather than accuracy, since inference is only 0.53 ms of the 53.4 ms loop.

Deployment posture. Given these bounds we position **BISINDO Bridge** as a communication aid for familiar partners — classrooms, family calls, small team meetings — where a signer can repeat or fingerspell after a rare misrecognition, not as an interpreter replacement. The sentence builder’s edit gestures exist precisely because the human stays in the loop: a fist deletes, a palm clears, and the conversation continues without breaking the meeting.

Limitations. Table 5 summarizes the eight limitations that bound deployment claims.

8 Conclusion and Future Work

BISINDO Bridge couples a dual-hand 84-D landmark 1D-CNN (98.45% Acc/F1, 61,274 parameters) with on-device ONNX inference and a full-mesh WebRTC meeting that broadcasts composed sentences. Every stage of recognition runs inside the participant’s browser, so a video meeting gains sign-language support without installs, accounts, or server-side vision.

Two results anchor the contribution. On the recognition side, a compact per-frame classifier is enough for the alphabet when the input is normalized geometry: three-stage normalization over an 84-D two-hand representation reaches 98.45% on held-out samples while consuming 0.53 ms of a 53.4 ms frame budget. On the systems side, the honest finding is that the media path, not model accuracy, is what limits real rooms: reception degrades asymmetrically as participants join, which we document through live failure modes rather than hide.

The design choices that make the tool usable in practice — browser-side privacy, ICE-restart recovery, two-hand letter coverage — required accepting a small accuracy cost against the single-hand baseline, and we believe that trade is correct: a communication tool for BISINDO must accept both hands.

Future work follows directly from the evaluation. For recognition, the per-frame model is the foundation for *continuous* signing: a temporal layer over committed letters, a broader vocabulary, and language modeling can turn the alphabet into flowing phrases. For evaluation, the immediate next measurement is a leave-one-signer-out protocol to quantify signer dependence, followed by usability studies with deaf participants, which this paper deliberately does not claim. For transport, the full mesh should give way to a media-relaying SFU so that room size stops being an $O(n^2)$ constraint, with a named tunnel and input hardening closing the remaining production gaps.

References

1. F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "MediaPipe Hands: On-device Real-time Hand Tracking," *arXiv:2006.10214*, 2020.
2. R. Auziqni, "BISINDO Indonesian Sign Language Recognition Using MediaPipe Holistic and LSTM Deep Learning Model," Undergraduate thesis, Universitas Mercu Buana, Jakarta, Indonesia, 2022. <https://repository.mercubuana.ac.id/id/eprint/69873>, accessed 2026.
3. T. M. A. Afwan, R. Gernowo, and H. A. Wibawa, "Deep Learning-Based Recognition of Indonesian Sign Language (BISINDO) Alphabetic Gestures Using Skeletal Feature Extraction and LSTM," *Jurnal Teknik Informatika (Jutif)*, vol. 7, no. 2, 2026, doi: 10.52436/1.jutif.2026.7.2.5337.
4. A. Aljabar and Suharjito, "BISINDO (Bahasa Isyarat Indonesia) Sign Language Recognition Using CNN and LSTM," *Advances in Science, Technology and Engineering Systems Journal*, vol. 5, no. 5, pp. 282–287, 2020, doi: 10.25046/aj050535.
5. R. Rastgoo, K. Kiani, and S. Escalera, "Sign Language Recognition: A Deep Survey," *Expert Systems with Applications*, vol. 164, Art. no. 113794, 2021.
6. M. Al-Qurishi, T. Khalid, and R. Souissi, "Deep Learning for Sign Language Recognition: Current Techniques, Benchmarks, and Open Issues," *IEEE Access*, vol. 9, pp. 126917–126951, 2021.
7. M. Oudah, A. Al-Naji, and J. Chahl, "Hand Gesture Recognition Based on Computer Vision: A Review," *Journal of Imaging*, vol. 6, no. 8, Art. no. 73, 2020.
8. A. K. Fadzli and M. Rahardi, "Comparative Analysis of MobileNetV3 and EfficientNetv2B0 in BISINDO Hand Sign Recognition Using MediaPipe Landmarks," *Journal of Applied Informatics and Computing*, vol. 10, no. 1, 2026, doi: 10.30871/jaic.v10i1.11878.
9. C. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," *arXiv:1906.08172*, 2019.
10. N. Al Mudawi et al., "Innovative Healthcare Solutions: Robust Hand Gesture Recognition of Daily Life Routines Using 1D CNN," *Frontiers in Bioengineering and Biotechnology*, vol. 12, Art. no. 1401803, 2024, doi: 10.3389/fbioe.2024.1401803.
11. K. Kumar et al., "Mediapipe and CNNs for Real-Time ASL Gesture Recognition," *arXiv:2305.05296*, 2023.
12. Microsoft, "ONNX Runtime," documentation, <https://onnxruntime.ai>, accessed 2026.